EX. NO.: 6	IMPLEMENTATION OF N-QUEENS PROBLEM USING
DATE:	

AIM:

To develop a Python program that solves the N-Queens problem using the Backtracking algorithm by placing N queens on an N × N chessboard such that no two queens attack each other.

ALGORITHM:

Step 1: Start the program.
Step 2: Read the size of the chessboard N.
Step 3: Create an empty N × N chessboard.
Step 4: Define a function is_safe() to check whether a queen can be placed safely.
Step 5: Check the row on the left side for conflicts.
Step 6: Check the upper-left diagonal and lower-left diagonal for conflicts.
Step 7: Define a recursive backtracking function solve_n_queens().
Step 8: Place queens one column at a time**.**
Step 9: If a conflict occurs, remove the queen (backtrack) and try the next row.
Step 10: End the program.

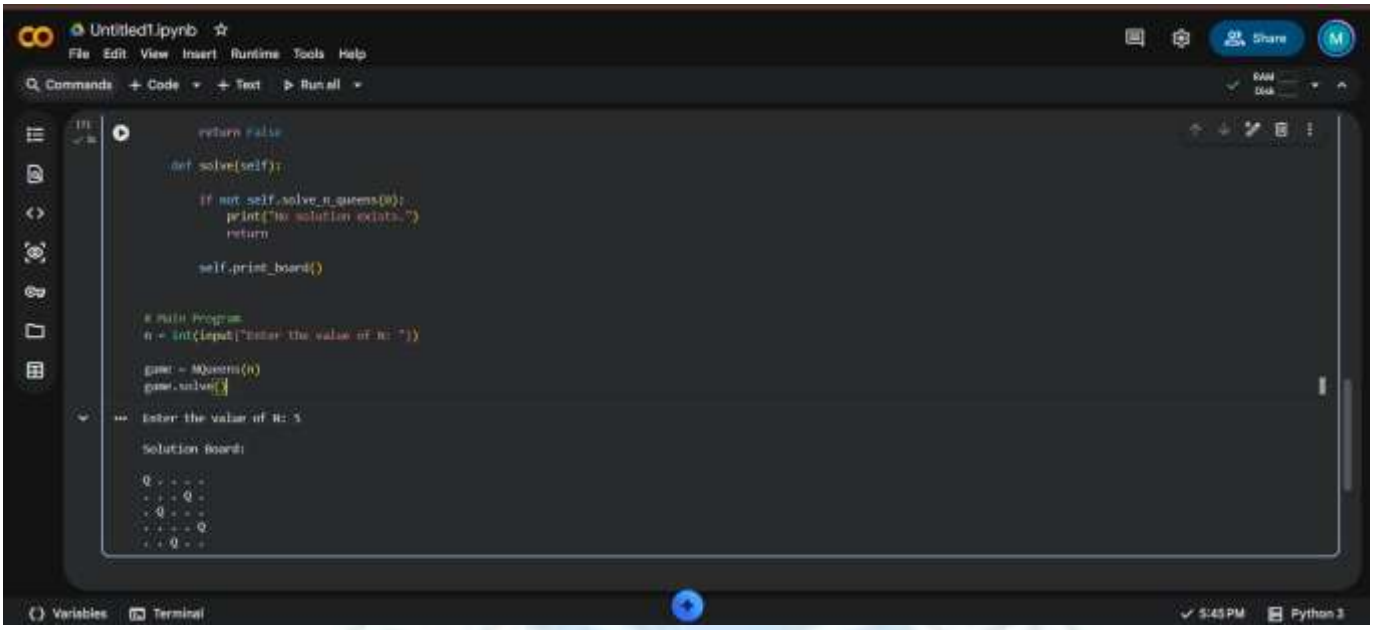
PROGRAM:

```
class NQueens:
    def __init__(self, n):
        self.N = n
        self.board = [[0 for _ in range(n)] for _ in range(n)]
    def print_board(self):
        print("\nSolution Board:\n")
        for row in self.board:
            for cell in row:
                if cell == 1:
                    print("Q", end=" ")
                else:
                    print(".", end=" ")
            print()
    def is_safe(self, row, col):
        for i in range(col):
            if self.board[row][i] == 1:
                return False
        i = row
        j = col
        while i >= 0 and j >= 0:
            if self.board[i][j] == 1:
```

```
                return False
            i -= 1
            j -= 1
        # Check lower-left diagonal
        i = row
        j = col

        while i < self.N and j >= 0:
            if self.board[i][j] == 1:
                return False
            i += 1
            j -= 1
        return True
    def solve_n_queens(self, col):
        if col >= self.N:
            return True
        for row in range(self.N):
            if self.is_safe(row, col):
                self.board[row][col] = 1
                if self.solve_n_queens(col + 1):
                    return True
                self.board[row][col] = 0
        return False
    def solve(self):
        if not self.solve_n_queens(0):
            print("No solution exists.")
            return
        self.print_board()
n = int(input("Enter the value of N: "))
game = NQueens(n)
game.solve()
```

OUTPUT:



RESULT:

Thus, the N-Queens problem was successfully implemented using the Backtracking technique in Python, and a valid arrangement of N queens on the chessboard was obtained without any conflicts.

EX. NO.: 1	IMPLEMENTATION OF TIC-TAC-TOC GAME
DATE:	

AIM:

To write and implement a Python program for a Tic-Tac-Toe game where two players can play alternatively by entering their moves.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a 3 × 3 Tic-Tac-Toe board with empty spaces.

Step 3: Display the board.

Step 4: Assign X to Player 1 and O to Player 2.

Step 5: Ask the current player to enter a position from 1 to 9.

Step 6: Check whether the selected position is empty.

Step 7: Place the player's symbol in the selected position.

Step 8: Check the rows, columns, and diagonals for a winning combination.

Step 9: If a player gets three consecutive symbols, declare that player as the winner.

Step 10: If all positions are filled without a winner, declare the game as a draw.

Step 11: Otherwise, switch to the other player and repeat the steps.

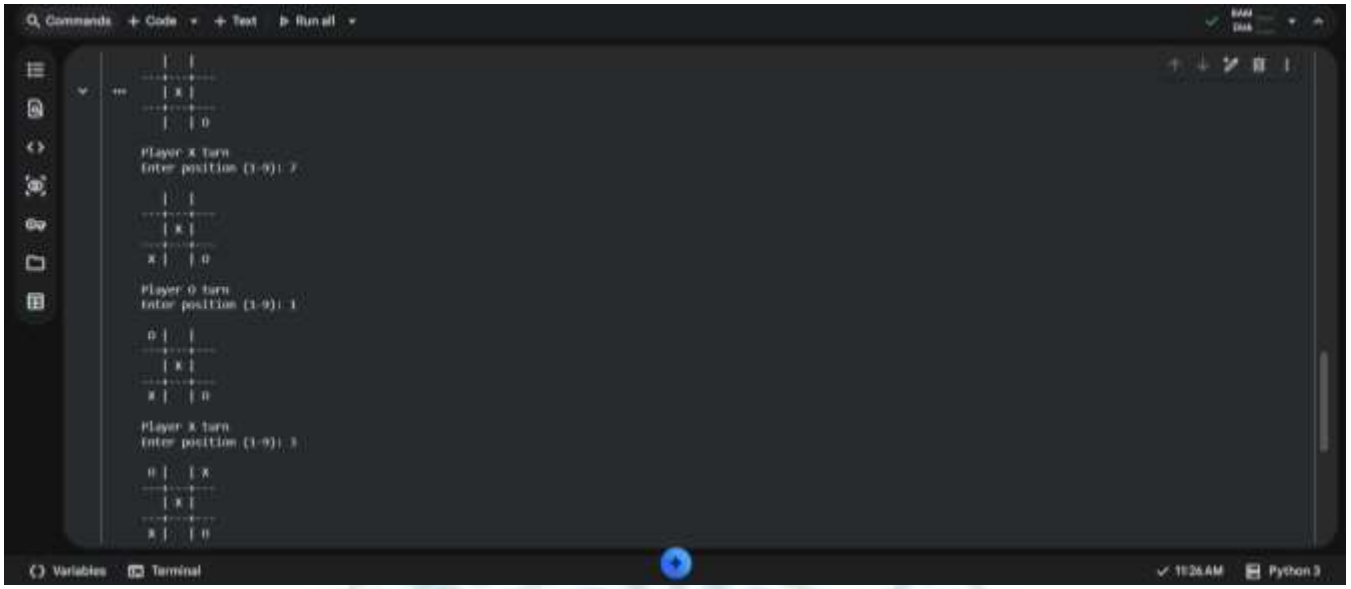
Step 12: Stop the program.

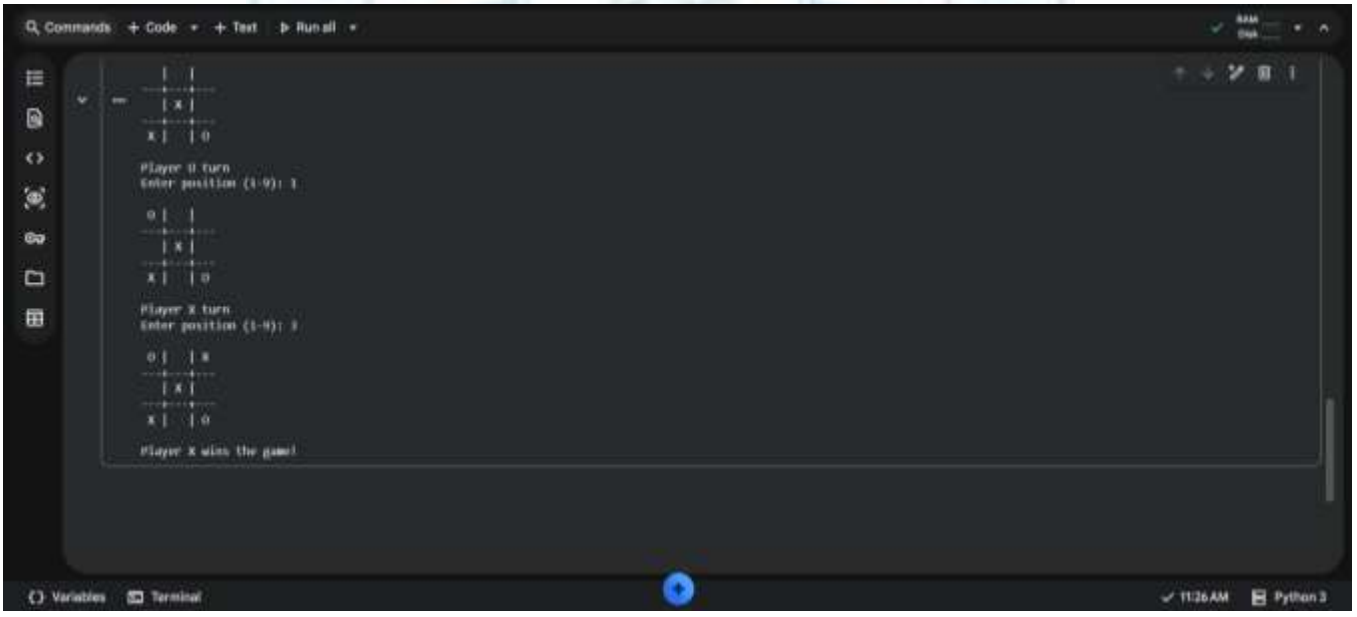
PROGRAM:

```
board = [' '] * 9
def display_board():
    print()
    print(" {} | {} | {} ".format(board[0], board[1], board[2]))
    print("---+---+---")
    print(" {} | {} | {} ".format(board[3], board[4], board[5]))
    print("---+---+---")
    print(" {} | {} | {} ".format(board[6], board[7], board[8]))
    print()
def check_winner(player):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
```

```
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
def tic_tac_toe():
    player = 'X'
    for turn in range(9):
        display_board()
        print("Player", player, "turn")
        while True:
            try:
                position = int(input("Enter position (1-9): "))
                if position < 1 or position > 9:
                    print("Please enter a number between 1 and 9.")
                elif board[position - 1] != ' ':
                    print("Position already occupied. Choose another.")
                else:
                    break
            except ValueError:
                print("Enter a valid number.")
        board[position - 1] = player
        if check_winner(player):
            display_board()
            print("Player", player, "wins the game!")
            return
        if turn == 8:
            display_board()
            print("The game is a draw!")
            return
        player = 'O' if player == 'X' else 'X'
tic_tac_toe()
```

OUTPUT:


Commands
+ Code
+ Text
Run all
| |
---+---+---
| X |
---+---+---
| | O
Player X turn
Enter position (1-9): 7
| |
---+---+---
| X |
---+---+---
X | | O
Player O turn
Enter position (1-9): 1
O | |
---+---+---
| X |
---+---+---
X | | O
Player X turn
Enter position (1-9): 3
O | | X
---+---+---
| X |
---+---+---
X | | O
Variables
Terminal
11:26 AM
Python 3


Commands
+ Code
+ Text
Run all
| |
---+---+---
| X |
---+---+---
X | | O
Player O turn
Enter position (1-9): 1
O | |
---+---+---
| X |
---+---+---
X | | O
Player X turn
Enter position (1-9): 3
O | | X
---+---+---
| X |
---+---+---
X | | O
Player X wins the game!
Variables
Terminal
11:26 AM
Python 3



RESULT:

Thus, the Tic-Tac-Toe game was successfully implemented using Python. The program allows two players to enter their moves, checks for a winning combination, and displays the winner or draw result at the end of the game.

EX. NO.: 2	IMPLEMENTATION OF ALPHA-BETA PRUNING
DATE:	

AIM:

To write and implement a Python program to perform **Alpha-Beta Pruning**, an optimization technique used with the Minimax algorithm to reduce the number of nodes evaluated in a game tree.

ALGORITHM:

Step 1: Start the program.

Step 2: Define the game tree and assign values to its leaf nodes.

Step 3: Initialize alpha to negative infinity and beta to positive infinity.

Step 4: Apply the Minimax algorithm with Alpha-Beta Pruning.

Step 5: For a MAX node, find the maximum value of its child nodes and update alpha.

Step 6: For a MIN node, find the minimum value of its child nodes and update beta.

Step 7: If alpha >= beta, prune the remaining branches because they do not affect the final result.

Step 8: Continue evaluating the remaining required nodes.

Step 9: Return the optimal value obtained from the root node.

Step 10: Display the optimal value.

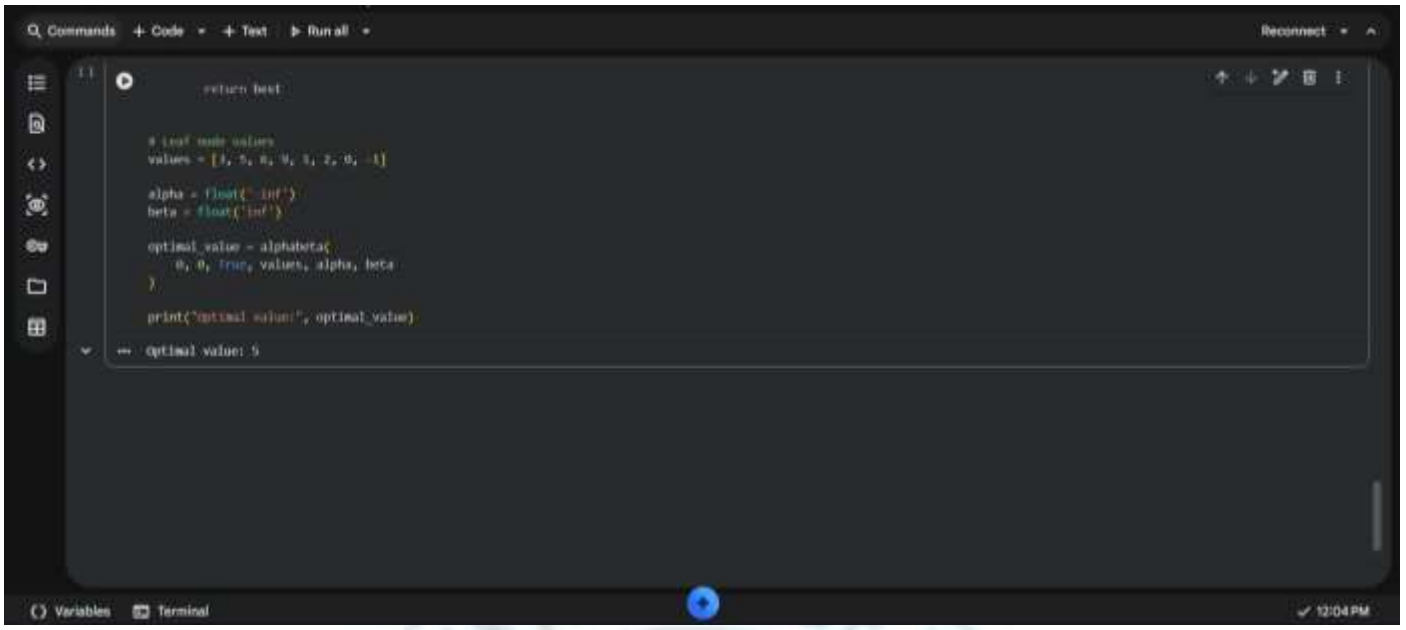
Step 11: Stop the program.

PROGRAM:

```
def alphabeta(depth, node, maximizing_player, values, alpha, beta):
 if depth == 3:
    return values[node]
 if maximizing_player:
    best = float('-inf')
    for i in range(2):
       value = alphabeta(
          depth + 1,
          node * 2 + i,
          False,
          values,
          alpha,
          beta
```

```
 best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = float('inf')
        for i in range(2):
            value = alphabeta(
                depth + 1,
                node * 2 + i,
                True,
                values,
                alpha,
                beta
            )
            best = min(best, value)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best
values = [3, 5, 6, 9, 1, 2, 0, -1]
alpha = float('-inf')
beta = float('inf')
optimal_value = alphabeta(
    0, 0, True, values, alpha, beta
)
print("Optimal value:", optimal_value)
```

OUTPUT:



RESULT:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented using Python. The program evaluates the game tree efficiently by pruning unnecessary branches and determines the optimal value as 5.

EX. NO.: 3	IMPLEMENTATION OF MONKEY BANANA PROBLEM
DATE:	

AIM:

To write and implement a Python program to solve the Monkey Banana Problem using the concept of Artificial Intelligence problem-solving and state-space search.

ALGORITHM:

Step 1: Start the program.

Step 2: Define the initial state of the monkey, box, and banana.

Step 3: Check whether the monkey can reach the banana directly.

Step 4: If the banana is out of reach, the monkey moves the box to the position below the banana.

Step 5: The monkey pushes the box to the required position.

Step 6: The monkey climbs onto the box.

Step 7: The monkey reaches and grabs the banana.

Step 8: Display the sequence of actions performed by the monkey.

Step 9: Stop the program**.**

PROGRAM:

```
class MonkeyBananaProblem:
def __init__(self):
    # Initial state
    self.monkey_position = "door"
    self.box_position = "corner"
    self.banana_position = "center"
    self.monkey_on_box = False
    self.has_banana = False
    def display_state(self):
    print("\nCurrent State:")
    print("Monkey Position :", self.monkey_position)
    print("Box Position    :", self.box_position)
    print("Banana Position :", self.banana_position)
    print("Monkey on Box   :", self.monkey_on_box)
    print("Has Banana      :", self.has_banana)
```

```
def move_monkey(self, position):
    self.monkey_position = position
    print("\nAction: Monkey moves to", position)
def push_box(self, position):
    if self.monkey_position == self.box_position:
        self.box_position = position
        self.monkey_position = position
        print("Action: Monkey pushes the box to", position)
    else:
        print("Monkey cannot push the box.")
        print("Monkey must be near the box.")
def climb_box(self):
    if self.monkey_position == self.box_position:
        self.monkey_on_box = True
        print("Action: Monkey climbs onto the box.")
    else:
        print("Monkey cannot climb the box.")
        print("Monkey must be near the box.")
def grab_banana(self):
    if (self.monkey_on_box and
            self.monkey_position == self.banana_position and
            self.box_position == self.banana_position):
        self.has_banana = True
        print("Action: Monkey grabs the banana.")
    else:
        print("Monkey cannot grab the banana.")
        print("Monkey must be on the box below the banana.")
def check_goal(self):
    return self.has_banana
def solve(self):
    print("========================================")
    print("       MONKEY BANANA PROBLEM")
    print("========================================")
    print("\nInitial State:")
    self.display_state()
    print("\n--------- SOLUTION ---------")
    self.move_monkey(self.box_position)
    self.push_box(self.banana_position)
    self.climb_box()
    self.grab_banana()
```

```
    print("\n--------- FINAL STATE ---------")
        self.display_state()
        if self.check_goal():
            print("\nGoal Achieved!")
            print("Monkey successfully gets the banana.")
        else:
            print("\nGoal Not Achieved.")
            print("Monkey could not get the banana.")
     problem = MonkeyBananaProblem()
      problem.solve()
```

OUTPUT:



RESULT:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented using Python. The program evaluates the game tree efficiently by pruning unnecessary branches and determines the optimal value as 5.

EX. NO.: 4	CONSTRUCTION OF AND-OR GRAPH
DATE:	

AIM:

To write and implement a Python program to construct an AND-OR graph from a given knowledge base and represent the relationships between facts and their sub-goals.

ALGORITHM:

Step 1: Start the program.

Step 2: Define the knowledge base as a set of facts and rules.

Step 3: Represent each rule as a relationship between a goal and its sub-goals.

Step 4: Identify the main goal from the knowledge base.

Step 5: If a goal has multiple sub-goals connected by AND, all sub-goals must be satisfied.

Step 6: If a goal has alternative sub-goals connected by OR, any one of them can be satisfied.

Step 7: Construct the AND-OR graph using the defined relationships.

Step 8: Display the nodes and their AND/OR relationships.

Step 9: Stop the program.

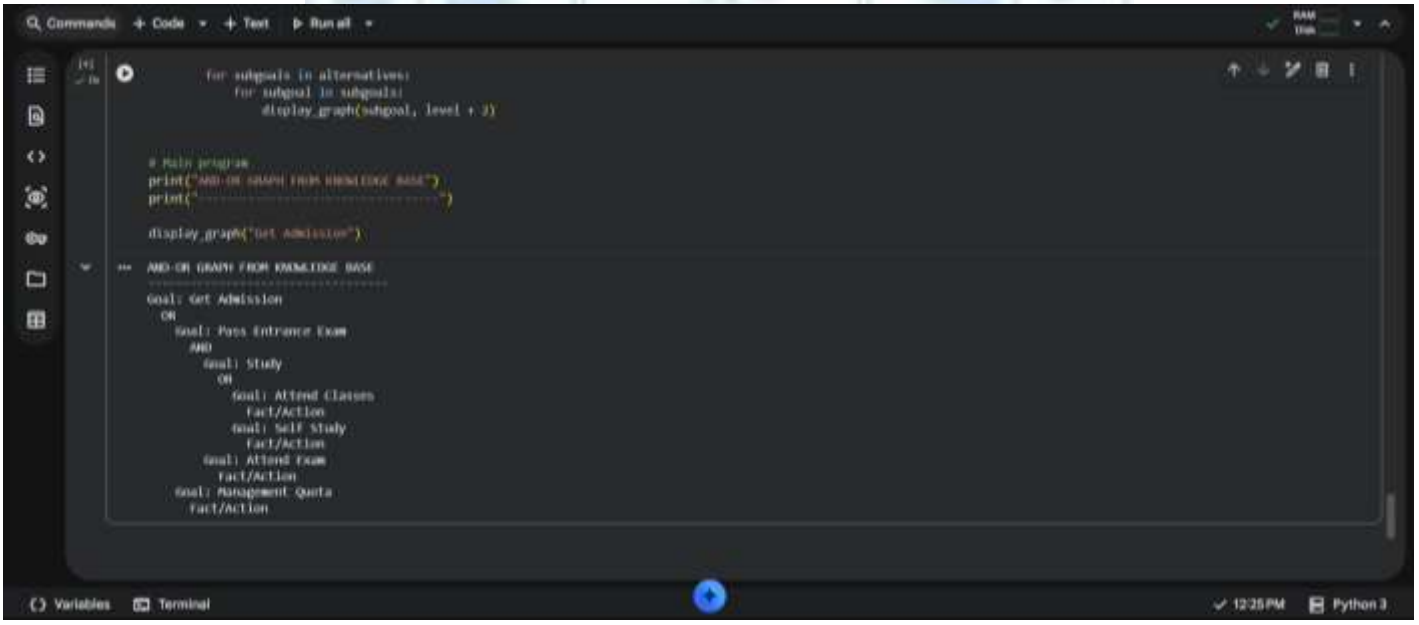
PROGRAM:

```
knowledge_base = {
"Get Admission": {
   "OR": [
      ["Pass Entrance Exam"],
      ["Management Quota"]
   ]
},
"Pass Entrance Exam": {
   "AND": [
      ["Study"],
      ["Attend Exam"]
   ]
},
"Study": {
   "OR": [
      ["Attend Classes"],
      ["Self Study"]
   ]
```

```
            }
        }
        def display_graph(goal, level=0):
            print("  " * level + "Goal:", goal)
            if goal not in knowledge_base:
                print("  " * (level + 1) + "Fact/Action")
                return
            rules = knowledge_base[goal]
            for operator, alternatives in rules.items():
                print("  " * (level + 1) + operator)
                for subgoals in alternatives:
                    for subgoal in subgoals:
                        display_graph(subgoal, level + 2)
        print("AND-OR GRAPH FROM KNOWLEDGE BASE")
        print("----------------------------------------")

display_graph("Get Admission")
```

OUTPUT:



RESULT:

Thus, the AND-OR graph was successfully constructed from the given knowledge base using Python. The program represents AND relationships, where all sub-goals are required, and OR relationships, where any one alternative can satisfy the goal.

EX. NO.: 5	IMPLEMENTATION OF SIMPLE CHATBOT
DATE:	

AIM:

To write and implement a simple chatbot using Python that interacts with the user and provides suitable responses based on the user's input.

ALGORITHM:

Step 1: Start the program.
Step 2: Define a function named technical_chatbot().
Step 3: Display the chatbot welcome message and instructions.
Step 4: Create a dictionary (knowledge_base) containing technical questions.
Step 5: Start an infinite loop to continuously interact with the user.
Step 6: Accept a question from the user and convert it to lowercase.
Step 7: Search for the user's question in the knowledge_base dictionary.
Step 8: Retrieve the corresponding answer; if the question is not found, display a default response.
Step 9: Print the chatbot's response and check whether the user entered “bye” terminate the loop.
Step 10: End the program.

PROGRAM:

```
def technical_chatbot():

    print("=" * 60)
    print("       TECHNICAL ASSISTANT CHATBOT")
    print("=" * 60)
    print("Ask questions about Python, AI, ML, SQL, Cloud,")
    print("Cyber Security, Web Development, Data Science, etc.")
    print("Type 'exit' to end the conversation.")
    print("=" * 60)

    knowledge_base = {

        "hello":
        "Hello! I am a Technical Assistant Chatbot.",

        "who are you":
        "I am a Python-based chatbot designed to answer technical questions.",

        "what is python":
        "Python is a high-level, interpreted programming language known for its simplicity and versatility.",

        "what is ai":
        "Artificial Intelligence is the simulation of human intelligence in machines.",

        "what is machine learning":
        "Machine Learning is a branch of AI that enables systems to learn from data without explicit programming.",

        "what is deep learning":
        "Deep Learning is a subset of Machine Learning that uses multi-layer neural networks.",
```

```
"what is data science":
"Data Science combines statistics, programming and domain knowledge to extract insights from data.",

"what is sql":
"SQL stands for Structured Query Language and is used to manage databases.",

"what is database":
"A database is an organized collection of structured information or data.",

"what is cloud computing":
"Cloud Computing provides computing resources such as servers and storage over the internet.",

"what is cybersecurity":
"Cybersecurity protects computer systems, networks and data from digital attacks.",

"what is an algorithm":
"An algorithm is a step-by-step procedure used to solve a problem.",

"what is a data structure":
"A data structure is a way of organizing and storing data efficiently.",

"what is oops":
"Object-Oriented Programming is a programming paradigm based on objects and classes.",

"what is inheritance":
"Inheritance allows a class to acquire properties and methods from another class.",

"what is polymorphism":
"Polymorphism allows the same interface to be used for different data types.",

"what is encapsulation":
"Encapsulation binds data and methods together into a single unit.",

"what is abstraction":
"Abstraction hides implementation details and shows only essential features.",

"what is html":
"HTML is the standard markup language used to create web pages.",

"what is css":
"CSS is used to style and format web pages.",

"what is javascript":
"JavaScript is a scripting language used to create interactive web pages.",

"what is react":
"React is a JavaScript library for building user interfaces.",

"what is django":
"Django is a high-level Python web development framework.",

"what is api":
"API stands for Application Programming Interface and allows software systems to communicate.",

"what is git":
"Git is a distributed version control system used to track code changes.",

"what is github":
"GitHub is a cloud platform used to host and manage Git repositories.",
```

```
        "what is linux":
        "Linux is an open-source operating system widely used in servers and development.",

        "what is operating system":
        "An Operating System manages hardware resources and provides services to software.",

        "what is computer network":
        "A computer network is a group of interconnected devices that share resources.",

        "what is internet":
        "The Internet is a global network connecting millions of computers worldwide.",

        "what is iot":
        "IoT stands for Internet of Things, where devices communicate through the internet.",

        "what is blockchain":
        "Blockchain is a decentralized and distributed ledger technology.",

        "what is generative ai":
        "Generative AI creates new content such as text, images and code.",

        "what is chatbot":
        "A chatbot is a software application that simulates human conversation.",

        "thank you":
        "You are welcome. Happy Learning!",

        "bye":
        "Goodbye! Have a great day."
    }

    while True:
        question = input("\nYou : ").lower()

        response = knowledge_base.get(
            question,
            "Sorry, I do not have information on that topic."
        )

        print("Bot :", response)

        if question == "bye":
            break


technical_chatbot()
```

OUTPUT:



RESULT:

Thus, the simple chatbot was successfully implemented using Python. The chatbot accepts user messages, searches for suitable predefined responses, and provides an appropriate reply.